	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No:92301733038**

Aim: To obtain the classification of various classes using k-Nearest Neighbor approach

IDE: Google Colab

Theory:

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line. Therefore, larger k value means smother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models. It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

The model representation for KNN is the entire training dataset. It is as simple as that. KNN has no model other than storing the entire dataset, so there is no learning required. Efficient implementations can store the data using complex data structures like k-d trees to make look-up and matching of new patterns during prediction efficient. Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

<table>
<tr><td></td><td colspan="2">Marwadi University
Faculty of Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Artificial Intelligence (01CT0616)</td><td colspan="2">Aim: To obtain the classification of various classes using k-Nearest Neighbor approach</td></tr>
<tr><td>Experiment No: 03</td><td>Date:</td><td>Enrollment No:92301733038</td></tr>
</table>

Methodology:

1. **Load the basic libraries and packages**

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

2. **Load the dataset**

Code:

```
url = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"
df = pd.read_csv(url)
```

3. **Analyse the dataset**

Code:

```
print("First 5 rows:\n", df.head())
print("\nDataset Info:\n")
print(df.info())
print("\nStatistics:\n", df.describe())
```

Output:

```
First 5 rows:
    sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Dataset Info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

Statistics:
        sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.054000      3.758667     1.198667
std        0.828066     0.433594      1.764420     0.763161
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

4. **Pre-process the data**

Code:

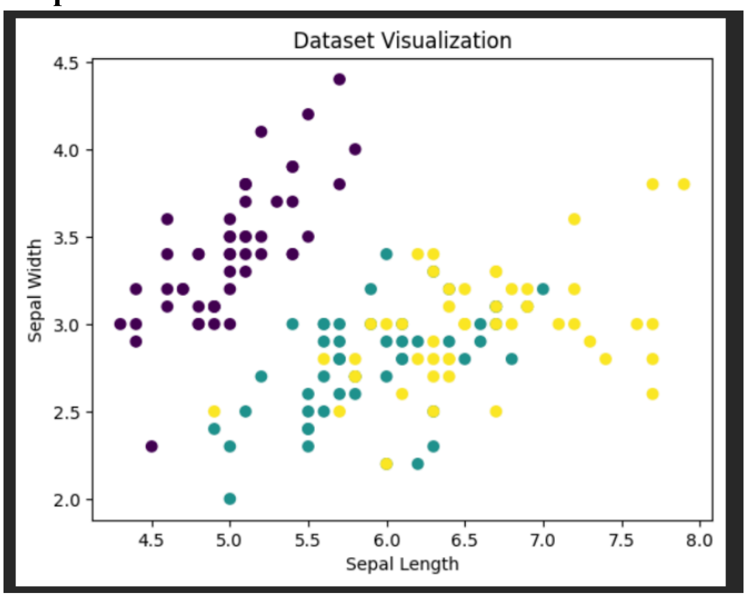
```
# Convert categorical labels into numbers
df['species'] = df['species'].astype('category').cat.codes
```

5. **Visualize the Data**

Code:

```
plt.scatter(df['sepal_length'], df['sepal_width'], c=df['species'])
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Dataset Visualization")
plt.show()
```

Output:



6. **Separate the feature and prediction value columns**

Code:

```
X = df.iloc[:, :-1].values   # all columns except last
y = df.iloc[:, -1].values    # last column (species)
# Split dataset into training and test manually
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

7. **Select the number K of the neighbors**

Code:

```
K = 5
```

8. **Calculate the Euclidean distance of K number of neighbors**

Code:

```
def euclidean_distance(x1, x2):
  return np.sqrt(np.sum((x1 - x2)**2))
```

9. **Take the K nearest neighbors as per the calculated Euclidean distance.**

Code:

```
def knn_predict(X_train, y_train, x_test, K):
    distances = []

    # Calculate distance from all training points
    for i in range(len(X_train)):
        dist = euclidean_distance(X_train[i], x_test)
        distances.append((dist, y_train[i]))
```

```
    # Sort based on distance
    distances.sort(key=lambda x: x[0])
```

10. **Among these k neighbors, count the number of the data points in each category.**

```
   # Take K nearest neighbors
    neighbors = distances[:K]

    # Count categories
    labels = [label for (_, label) in neighbors]
```

11. **Assign the new data points to that category for which the number of the neighbor is maximum.**

Code:

```
 # Majority voting
    prediction = Counter(labels).most_common(1)[0][0]

    return prediction
```

12. **Our model is ready.**

Code:

```
predictions = []

for i in range(len(X_test)):
    pred = knn_predict(X_train, y_train, X_test[i], K)
    predictions.append(pred)
```

```
# Accuracy calculation
accuracy = np.sum(predictions == y_test) / len(y_test)
print("\nModel Accuracy:", accuracy)
```

Output:

```
...
    Model Accuracy: 0.8
```

Pre Lab Exercise:

a. What is K in KNN-approach?

In K-Nearest Neighbors (KNN), **K** represents the number of nearest data points considered to make a prediction. The algorithm looks at the K closest neighbors and decides the output based on them.

b. Which are the types of distance metrices?

Common distance metrics include Euclidean distance, Manhattan distance, Minkowski distance, and Cosine similarity. These metrics help measure how close or similar data points are to each other.

c. What can be the criteria of selection of the value of K?

The value of K should be chosen carefully to balance accuracy. A small K may lead to overfitting, while a large K may cause underfitting. Usually, the best K is selected using methods like cross-validation or by testing different values.

d. What are the advantages of KNN algorithm?

KNN is simple and easy to understand, requires no training phase, and works well with small datasets. It can handle both classification and regression problems effectively.

Observation and Result Analysis:

a. Nature of the dataset

The dataset (Iris dataset) contains numerical features like sepal length, sepal width, petal length, and petal width, along with categorical labels representing different flower species. It is a well-structured dataset suitable for classification problems, where patterns between features and classes are clearly distinguishable.

b. During Training Process

KNN does not have a traditional training phase; it simply stores the training data. During prediction, it calculates distances between the test point and all training points, then selects the nearest neighbors based on the chosen value of K.

c. After the training Process

After applying KNN, the model predicts the class of test data based on majority voting among the nearest neighbors. The results generally show good accuracy when the value of K is properly selected and the data is well-scaled.

d. Observation over the Learning Curve

KNN typically does not use a learning curve like gradient-based models. However, performance can be analyzed by varying the value of K—small K may overfit, while large K may underfit, affecting overall accuracy.

Post Lab Exercise:

a. Is KNN useful in regression-based problems?

Yes, KNN can be used for regression. Instead of majority voting, it predicts the value by taking the average of the K nearest neighbors.

b. Why KNN is a non-parametric algorithm?

KNN is non-parametric because it does not assume any specific form or distribution of the data. It directly uses the training data to make predictions.

c. What are the assumptions of KNN approach?

KNN assumes that similar data points exist close to each other. It also assumes that the distance metric used can correctly measure similarity between data points.

d. How can the KNN approach make the prediction of the unseen dataset?

For a new data point, KNN calculates the distance from all training points, selects the K nearest neighbors, and predicts the output based on majority voting (classification) or averaging (regression).

e. Why is KNN called a Lazy Learner?

KNN is called a lazy learner because it does not learn a model during training. It simply stores the data and performs computation only when a prediction is required.

f. Why should we not use KNN for large dataset?

KNN becomes slow for large datasets because it has to calculate distances with all data points for every prediction. It also requires high memory to store all training data.